뉴토피아(Newtopia) 프로젝트 포팅 매뉴얼

목차

1. 프로젝트 개요
2. 시스템 환경 및 버전 정보
3. 빌드 시 환경 변수
4. 개발 환경 구축
5. 배포 환경 설정
6. 주요 계정 및 설정 파일
7. 배포 시 특이사항

---

1. 프로젝트 개요

뉴토피아는 뉴스 크롤링, 분석, 게임 서버 등을 포함하는 마이크로서비스 아키텍처 기반 프로젝트입니다.

주요 서비스 구성

- **Game Server**: 게임 로직 처리 (Spring Boot)
- **News Crawler**: 뉴스 크롤링 서비스 (Spring Boot)
- **Frontend**: 웹 클라이언트 (React + Vite)
- **Data Pipeline**: 데이터 처리 파이프라인 (Kafka Connect, Spark)
- **ML Workers**: 텍스트 분석 및 감정 분석 워커

개발을 위한 Quick Start

```
git clone https://lab.ssafy.com/s13-bigdata-dist-sub1/S13P21C203
cd S13P21C203
docker-compose up --build -d
```

---

2. 시스템 환경 및 버전 정보

2.1 JVM 및 런타임 환경

Game Server & News Crawler

- **JDK**: Amazon Corretto 17 (Alpine 3.18 기반)

- **WAS**: Spring Boot 3.5.5(Gameserver, Tomcat), Spring Boot 3.4.9 (news-crawler, Netty)
- **빌드 도구**: Gradle 8.5
- **베이스 OS**: Alpine Linux 3.18.4

```
FROM amazoncorretto:17-alpine3.18 AS builder-jre
FROM gradle:8.5-jdk17-alpine AS builder-app
FROM alpine:3.18.4 AS runtime
```

Frontend

- **Node.js**: 18.19.1 (Alpine 기반)
- **웹서버**: Nginx (Alpine 기반)
- **빌드 도구**: Vite 6.3.5
- **프레임워크**: React 19.0.0

```
FROM node:18.19.1-alpine AS builder
FROM nginx:alpine AS production
```

2.2 주요 프레임워크 및 라이브러리

Spring Boot 애플리케이션

- Spring Boot: 3.5.5, 3.4.9
- Spring Security
- Spring Data JPA
- Spring Kafka
- Swagger: 2.8.5

Frontend 애플리케이션

- React: 19.0.0
- React DOM: 19.0.0
- TanStack Router: 1.130.2
- TanStack Query: 5.66.5
- Styled Components: 6.1.19
- Zustand: 5.0.8
- Tailwind CSS: 4.0.6
- TypeScript: 5.7.2
- Recharts: 3.2.1

2.3 데이터베이스 및 미들웨어

- **PostgreSQL**: latest

- **Redis**: 6.2
- **Apache Kafka**: latest (KRaft 모드)
- **MinIO**: latest
- **Kafka Connect**: Confluent 7.5.0

2.4 모니터링 도구

- **Prometheus**: latest
- **Grafana**: latest

---

3. 빌드 시 환경 변수

3.1 공통 환경 변수

Docker Compose 환경 (개발)

```
# PostgreSQL
POSTGRES_USER=newtopia
POSTGRES_PASSWORD=newtopia
POSTGRES_DB=newtopia

# Redis (기본값)
REDIS_HOST=redis
REDIS_PORT=6379

# Kafka
KAFKA_BOOTSTRAP_SERVERS=kafka:9092

# MinIO
MINIO_ROOT_USER=minio
MINIO_ROOT_PASSWORD=minio123
S3_ENDPOINT=http://minio:9000
S3_ACCESS_KEY=minio
S3_SECRET_KEY=minio123
```

Kubernetes 환경 (운영)

```
# Database
POSTGRES_HOST=${POSTGRES_HOST}
POSTGRES_PORT=${POSTGRES_PORT}
POSTGRES_DATABASE=${POSTGRES_DATABASE}
POSTGRES_USER=${POSTGRES_USER}
POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
```

```
# Redis
REDIS_HOST=${REDIS_HOST}
REDIS_PORT=${REDIS_PORT}

# Kafka
KAFKA_BOOTSTRAP_SERVERS=${KAFKA_BOOTSTRAP_SERVERS}
KAFKA_CONSUMER_GROUP=${KAFKA_CONSUMER_GROUP}

# JWT
JWT_SECRET=${JWT_SECRET}

# MinIO
MINIO_ENDPOINT=${MINIO_ENDPOINT}
MINIO_ACCESS_KEY=${MINIO_ACCESS_KEY}
MINIO_SECRET_KEY=${MINIO_SECRET_KEY}
MINIO_BUCKET_NAME=${MINIO_BUCKET_NAME}
MINIO_PUBLIC_BASE_URL=${MINIO_PUBLIC_BASE_URL}

# API Keys
OPENAPI_SECRET_KEY=${OPENAPI_SECRET_KEY}
```

3.2 ML 워커 환경 변수

Analyzer Worker

```
# Kafka
KAFKA_BOOTSTRAP=kafka:9092
SRC_TOPIC=newtopia.news.clean
DST_TOPIC=newtopia.news.analyzed
DLQ_TOPIC=newtopia.news.clean.dlq
GROUP_ID=newtopia.analyzer.v1
AUTO_OFFSET_RESET=earliest

# MinIO
S3_ENDPOINT=http://minio:9000
S3_ACCESS_KEY=minio
S3_SECRET_KEY=minio123
S3_BUCKET=newtopia
S3_REGION=us-east-1

# Model Configuration
MODEL_NAME=sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
MIN_MAJOR_CONF="0.60"
MIN_MINOR_CONF="0.60"
CLASSIFY_MAX_WORKERS="4"

# NVIDIA GPU
```

```
NVIDIA_VISIBLE_DEVICES=all
NVIDIA_DRIVER_CAPABILITIES=compute,utility
```

Sentiment Worker

```
# Model Configuration
MODEL_NAME=snunlp/KR-FinBert-SC
USE_CUDA="true"
BATCH_SIZE="32"
MAX_CONTENT_LEN="4000"
MAX_SEQ_LEN_TOKENS="256"
SLIDING_STRIDE_TOKENS="128"

# Performance
TOKENIZERS_PARALLELISM="false"
OMP_NUM_THREADS="1"
PRODUCER_COMPRESSION="lz4"
```

---

4. 개발 환경 구축

4.1 사전 요구사항

- Ubuntu 22.04.5 LTS
- Windows 10, 11
- Docker 20.0+
- Docker Compose 2.0+
- Git

4.2 소스 코드 클론

```
git clone https://lab.ssafy.com/s13-bigdata-dist-sub1/S13P21C203
cd S13P21C203
```

4.3 개발 환경 실행

전체 서비스 실행

```
docker-compose up -d
```

개별 서비스 실행

```
# 데이터베이스 및 미들웨어만
docker-compose up -d postgres redis kafka minio

# 게임 서버만
docker-compose up -d gameserver

# 프론트엔드만
docker-compose up -d react-nginx

# 뉴스 크롤러만
docker-compose up -d crawling-server
```

4.4 서비스 접근 주소

- Frontend: http://localhost:80
- Game Server: http://localhost:8080
- News Crawler: http://localhost:18080
- Kafka UI: http://localhost:9090
- MinIO Console: http://localhost:9001
- Grafana: http://localhost:3001
- Prometheus: http://localhost:9091

---

5. 배포 환경 설정 (Kubernetes)

5.1 사전 요구사항

- Ubuntu 22.04.5 LTS
- Kubernetes 경량 클러스터(k3s)
- NFS 서버 (외부 또는 클러스터 내부)
- Helm (모니터링 스택 배포 시)

5.2 인프라 구성 요소 설치

필수 오퍼레이터 및 CSI 드라이버 설치

```
# 1. NFS CSI Driver 설치 (스토리지를 위한 필수 구성요소)
chmod +x kubernetes/infra/install-nfs-csi.sh
./kubernetes/infra/install-nfs-csi.sh

# 2. Cert-Manager 설치 (TLS 인증서 자동 관리)
chmod +x kubernetes/infra/cert-manager.sh
./kubernetes/infra/cert-manager.sh
```

```
# 3. Strimzi Kafka Operator 설치 (Kafka 클러스터 관리)
chmod +x kubernetes/infra/strimzi-manifest.sh
./kubernetes/infra/strimzi-manifest.sh

# 4. Let's Encrypt Issuer 설정 (SSL 인증서 발급)
kubectl apply -f kubernetes/infra/letsencrypt-issuer.yaml
```

주의사항:

- NFS CSI Driver는 NFS 기반 스토리지 사용을 위해 필수
- Strimzi Operator는 Kafka 클러스터를 CRD(Custom Resource Definition)로 관리
- Cert-Manager는 Let's Encrypt를 통한 자동 SSL 인증서 발급에 필요

5.3 네임스페이스 생성

```
# 필수 네임스페이스 생성 (배포 전 우선 실행)
kubectl apply -f kubernetes/namespace/newtopia-app.yaml      # 애플리케이션
kubectl apply -f kubernetes/namespace/newtopia-db.yaml       # 데이터베이스
kubectl apply -f kubernetes/namespace/newtopia-kafka.yaml    # 카프카
kubectl apply -f kubernetes/namespace/newtopia-minio.yaml    # MinIO
kubectl apply -f kubernetes/namespace/observability.yaml    # 모니터링
```

5.3 스토리지 설정

```
# NFS 스토리지 클래스 생성
kubectl apply -f kubernetes/storage/nfs-aws-storageclass.yaml
kubectl apply -f kubernetes/storage/nfs-kafka-storageclass.yaml

# tailscale 방식으로 네트워크 구성할 경우만
kubectl apply -f kubernetes/storage/nfs-tailscale-storageclass.yaml
```

5.5 미들웨어 배포

```
# PostgreSQL
kubectl apply -f kubernetes/middleware/postgres-workload.yaml

# Redis
kubectl apply -f kubernetes/middleware/redis-workload.yaml

# Kafka
kubectl apply -f kubernetes/middleware/kafka-workload.yaml

# MinIO
kubectl apply -f kubernetes/middleware/minio-workload.yaml
```

5.6 애플리케이션 배포

```
# Game Server 배포
kubectl apply -f kubernetes/app/gameserver-workload.yaml

# Frontend 배포
kubectl apply -f kubernetes/app/frontend-workload.yaml

# News Crawler 배포
kubectl apply -f kubernetes/app/crawlingserver-workload.yaml
```

5.7 데이터 파이프라인 배포

```
# Analyzer Worker
# GPU Node 필수
kubectl apply -f kubernetes/pipeline/analyzer-workload.yaml

# Kafka UI
kubectl apply -f kubernetes/pipeline/kafka-ui-workload.yaml
```

5.8 모니터링 스택 배포

```
# Prometheus + Grafana (Helm 사용)
helm repo add prometheus-community https://prometheus-
community.github.io/helm-charts
helm install monitoring prometheus-community/kube-prometheus-stack -f
kubernetes/monitoring/values.yaml
```

5.9 Ingress 설정

애플리케이션 Ingress

```
# Frontend 및 Game Server Ingress
kubectl apply -f kubernetes/app/gameserver-ing.yaml

# MinIO 이미지 서버 Ingress
kubectl apply -f kubernetes/app/image-ing.yaml
```

도메인 설정:

- 메인 애플리케이션: `j13c203.p.ssafy.io`
- 모니터링 (Grafana): `j13c203a.p.ssafy.io`
- MinIO 이미지 서버: `j13c203.p.ssafy.io/newtopia-img`

특이사항:

- Traefik Ingress Controller 사용
- Let's Encrypt를 통한 자동 TLS 인증서 발급
- HTTP to HTTPS 자동 리다이렉트 설정

---

6. 주요 계정 및 설정 파일

6.1 데이터베이스 접속 정보

개발 환경 (Docker Compose)

```
Host: localhost
Port: 5432
Database: newtopia
Username: newtopia
Password: newtopia
```

운영 환경 (Kubernetes)

```
Host: ${POSTGRES_HOST}
Port: ${POSTGRES_PORT}
Database: ${POSTGRES_DATABASE}
Username: ${POSTGRES_USER}
Password: ${POSTGRES_PASSWORD}
```

6.2 Redis 접속 정보

개발 환경

```
Host: localhost
Port: 6379
Password: (없음)
```

운영 환경

```
Host: ${REDIS_HOST}
Port: ${REDIS_PORT}
```

6.3 MinIO 접속 정보

개발 환경

```
Endpoint: http://localhost:9000
Access Key: minio
Secret Key: minio123
Bucket: newtopia
Console: http://localhost:9001
```

6.4 Kafka 접속 정보

개발 환경

```
Bootstrap Servers: localhost:9092
External Port: localhost:29092
```

6.5 모니터링 접속 정보

Grafana

```
개발 환경: http://localhost:3001
운영 환경: https://j13c203a.p.ssafy.io
Username: admin
Password: admin
```

Prometheus

```
개발 환경: http://localhost:9091
```

6.6 주요 설정 파일 위치

Game Server

- `gameserver/src/main/resources/application.yml` - 기본 설정
- `gameserver/src/main/resources/application-docker.yml` - Docker 환경 설정
- `gameserver/src/main/resources/application-prod.yml` - 운영 환경 설정
- `gameserver/build.gradle` - 빌드 설정

News Crawler

- `news-crawler/src/main/resources/application.yml` - 기본 설정
- `news-crawler/src/main/resources/application-docker.yml` - Docker 환경 설정
- `news-crawler/src/main/resources/application-prod.yml` - 운영 환경 설정
- `news-crawler/build.gradle` - 빌드 설정

Frontend

- `frontend/package.json` - 패키지 설정
- `frontend/nginx.conf` - Docker 전용 Nginx 설정
- `frontend/ingix.prod.conf` - 배포 전용 Nginx 설정

Kubernetes

- `kubernetes/infra/` - 인프라 구성요소 설치 스크립트 (NFS CSI, Strimzi, Cert-Manager)
- `kubernetes/namespace/` - 네임스페이스 설정
- `kubernetes/app/` - 애플리케이션 워크로드 및 Ingress 설정
- `kubernetes/monitoring/values.yaml` - 모니터링 스택 설정
- `kubernetes/storage/` - 스토리지 관련 설정
- `kubernetes/middleware/` - 미들웨어 워크로드 설정
- `kubernetes/pipeline/` - 데이터 파이프라인 워크로드 설정

---

7. 배포 시 특이사항

7.1 Docker 관련 특이사항

멀티 스테이지 빌드

- Java 애플리케이션은 3단계 빌드 사용
  1. JRE 최적화 (jlink 사용)
  2. 애플리케이션 빌드 (Gradle)
  3. 런타임 이미지 생성

GPU 지원

```yaml
# ML 워커는 NVIDIA GPU 필요
gpus: all
environment:
  NVIDIA_VISIBLE_DEVICES: all
  NVIDIA_DRIVER_CAPABILITIES: compute,utility
```

사용자 권한

```yaml
# ML 워커는 non-root 사용자로 실행
user: "1000:1000"
```

7.2 Kubernetes 관련 특이사항

스토리지

- NFS 기반 스토리지 클래스 사용
- 각 서비스별 전용 PVC 생성
- Prometheus: 50Gi, Grafana: 10Gi, AlertManager: 10Gi

모니터링

- Prometheus Operator를 통한 모니터링 스택 관리
- ServiceMonitor를 통한 애플리케이션 메트릭 수집
- Grafana는 NodePort 30300으로 노출

네트워킹

- Traefik Ingress Controller 사용
- TLS 인증서 자동 관리
- 도메인별 라우팅 설정

7.3 보안 관련 특이사항

환경 변수 보안

- 운영 환경의 모든 민감 정보는 환경 변수로 주입
- JWT 시크릿, DB 패스워드, API 키 등은 별도 관리 필요

쿠키 설정 (운영 환경)

```yaml
cookie:
  secure: true
  same-site: "Lax"
```

7.4 성능 최적화

JVM 설정

```dockerfile
ENTRYPOINT ["java", "-Duser.timezone=Asia/Seoul", "-jar", "app.jar"]
```

데이터베이스 커넥션 풀

```yaml
hikari:
  maximum-pool-size: 20
  minimum-idle: 10
  connection-timeout: 30000
  idle-timeout: 600000
  max-lifetime: 1800000
```

Kafka Producer 설정

```yaml
producer:
  acks: all
  retries: 5
  properties:
    enable.idempotence: true
    compression.type: zstd
    linger.ms: 10
    batch.size: 131072
```

7.5 데이터 파이프라인 특이사항

Strimzi Kafka Operator

- **운영 방식**: Kubernetes 네이티브 Kafka 클러스터 관리를 위해 Strimzi Operator 사용
- **설치 위치**: `newtopia-kafka` 네임스페이스에 Operator 설치
- **관리 방식**: CRD(Custom Resource Definition)를 통한 선언적 Kafka 클러스터 관리
- **장점**:
  - Kubernetes 환경에서 Kafka 클러스터의 생성, 업데이트, 스케일링 자동화
  - ConfigMap을 통한 설정 관리
  - 모니터링 및 로깅 통합

Kafka Connect

- Confluent Hub를 통한 JDBC 커넥터 설치
- PostgreSQL JDBC 드라이버 추가 설치

ML 워커

- HuggingFace 모델 캐시 볼륨 마운트 필요
- 토큰화 병렬 처리 비활성화 (`TOKENIZERS_PARALLELISM=false`)
- OpenMP 스레드 수 제한 (`OMP_NUM_THREADS=1`)